Monitoring your models

The ML lifecycle does not end at deployment. Once a model is in production, we want to monitor the input data and output results of the model. In *Chapter 4*, we explored two key features of Databricks Lakehouse Monitoring integrated with Unity Catalog: Snapshot and TimeSeries profiles. Snapshot profiles are designed to provide an overview of a dataset at a specific point in time, capturing its current state. This is particularly useful for identifying immediate data quality issues or changes. On the other hand, TimeSeries profiles focus on how data evolves over time, making them ideal for tracking trends, patterns, and gradual changes in data distributions.

Expanding on these capabilities, Databricks also provides an Inference profile, tailored for monitoring machine learning models in production. This advanced profile builds upon the concept of TimeSeries profiles, adding critical functionalities for comprehensive model performance evaluation. It includes model quality metrics, essential for tracking the accuracy and reliability of predictions over time. It also records predictions and, optionally, ground truth labels, directly comparing expected and actual outcomes. This functionality is key for identifying model drift, where shifts in input data or the relationship between inputs and outputs occur.

Inference Tables in Databricks further bolster this monitoring capability. They contain essential elements such as model predictions, input features, timestamps, and potentially ground truth labels. Building a monitor on top of InferenceTables with the corresponding `InferenceLog` allows us to monitor model performance and data drift continuously.

In the event of drift detection, immediate actions should be taken – data pipeline verification or model retraining and evaluation are recommended. These steps ensure the model adapts to new data patterns, maintaining accuracy and effectiveness. Continuous monitoring against your baseline and cross-model versions is a strategy to adopt when trying to ensure a stable process across various deployed solutions.

Figure 8.1 is a code sample for creating an Inference monitor with model quality metrics using the `InferenceLog` profile type. This illustrates a practical application of this monitoring setup. We specify the `schedule` argument to make sure that this monitor is refreshed hourly.